

LeadSync — MRL Pacing & Off-Pace Flagging System (Build Spec)

Objective

Catch clients who are off-pace to hit their **minimum MRL threshold** for their current billing cycle *before* the cycle ends, so we can intervene mid-cycle instead of discovering a miss (and refund/guarantee risk) at the end. This is effectively an early-warning system for guarantee risk.

1. Core Pace Logic (pace-based, not a flat threshold)

A static “below X MRLs” check fires either too early (day 7) or too late (day 28). Instead:

- **Expected MRLs to date** = (business days elapsed in cycle ÷ total business days in cycle) × minimum MRL threshold.
- Compare **expected vs. actual MRLs delivered** so far. If actual is meaningfully behind expected pace, flag it.
- **Grace period**: do not evaluate or flag any client until they are at least **7 days into the billing cycle**, regardless of how far behind they’d technically be. Early-cycle numbers are noise.
- **Pace on business days, not calendar days** (we send Mon–Fri; calendar pacing skews clients to look behind because weekends are always zero-send).

Recoverability definition (used for tiering)

- **Max recent velocity** = the client’s best 7-day rolling MRL rate, expressed as MRLs per business day.
 - **Projected end-of-cycle total** = actual MRLs to date + (max recent velocity × business days remaining).
 - A client is **recoverable** if projected total ≥ threshold; **unrecoverable** if it falls short even at that best recent rate.
-

2. Severity Tiers

Each client is bucketed fresh on every job run:

- **On Track** — at or ahead of expected pace.
 - **At Risk** — behind pace but still mathematically recoverable at current/best velocity.
(*Proposed band: 10–30% behind expected pace. Confirm.*)
 - **Critical** — behind pace AND unrecoverable even at max recent velocity given business days remaining. (*Proposed: >30% behind pace, OR projected end-of-cycle total < threshold.*)
 - **Hard Critical override:** if ≤ 3 business days remain and the client is still below threshold, force Critical regardless of the math — there's no realistic time left to work new leads.
-

3. Dashboard Panel — “Clients Off-Pace for MRL Threshold”

Fully automatic. No manual Unreviewed / In Progress / Resolved workflow. Severity recalculates every run; clients move, downgrade, drop off, and reappear on their own with no reset logic.

Tier order (top to bottom) and collapse behavior

1. **Critical** — collapsible, **default expanded** (always open on load).
2. **At Risk** — collapsible, **default collapsed**.
3. **On Track** — collapsible, **default collapsed** (never auto-shows; available for audit only).

Within each tier, sort **worst pace-gap first**.

Auto-resolution behavior

- As a client's actual MRLs catch up to expected pace, severity downgrades (Critical → At Risk → On Track) and they move down the list.
- Once **On Track**, they drop off the active view entirely — no manual resolving.
- If they slip behind again later in the same cycle, they reappear/re-flag automatically.

Clean layout (avoid clutter)

- **Collapsed tier header** shows only a colored pill + count (e.g., “ Critical — 3”).
Nothing else.

- **Each client row = one compact summary line**, expandable for full detail:
 - Client name
 - Actual vs. expected MRLs (e.g., 6 / 11)
 - % behind pace
 - Projected end-of-cycle total vs. threshold
 - Auto-tagged likely root cause + confidence
 - Days left in cycle · days in current severity
 - **7-day velocity shown as a small sparkline** (up / flat / down) rather than a text block.
 - Consistent color language: red = Critical, amber = At Risk, green = On Track.
 - Expanding a row reveals the full diagnostic breakdown (section 4) so the default view stays scannable.
-

4. Root-Cause Diagnostic Engine

When a client flags At Risk or Critical, the job runs an ordered diagnostic against our current stack and returns **one auto-tagged likely cause + a confidence level (Medium / High)**. Checks are ordered by how often each is the actual culprit; the first strong hit wins because it can invalidate everything downstream.

Order of checks:

1. **Insufficient contact volume (check first)**. Total contacts across all their campaigns, B2B + B2C combined. If under ~25,000, this is almost always the root cause and nothing downstream matters.
 - Tag: `Insufficient contact volume (X loaded, <25K)`
2. **Infrastructure / healthy sending capacity**. Healthy vs. unhealthy sender account count/ratio for the client's tag. Cross-reference the self-reviving / infrastructure automation — is it actively reviving disconnected accounts, or are accounts stuck? Also surface known Missing MX / disconnected accounts we already track.
 - Tag: `Low healthy sending capacity (X/Y healthy)`
3. **Nurture routing gap**. Confirm automatic nurture is ON for the client's tag grouping, campaigns exist in the **correct EmailBison instance**, and routing matches on campaign

type + lead type + **exact tag** only. An unconfigured grouping or a campaign missing from the right instance is a silent leak.

- Tag: `Nurture routing gap`

4. **Failed campaigns.** Detect any failed campaigns on the account (see section 5).

- Tag: `Failed campaign(s) detected`

5. **Leads in campaign but not converting.** If contact volume is fine, infra is healthy, routing is clean, and no failed campaigns — but MRLs are still behind — the pipeline exists but isn't producing. This is the "work the lead / copy / timing" bucket, not a systems bug. Surface leads currently in campaigns (active pipeline that could still convert before cycle end).

- Tag: `Leads flowing, low conversion`

Confidence level (derived, not guessed)

- **High** — one signal is clearly out of range and the others are clean (e.g., 12K contacts loaded → obvious).
- **Medium** — two signals are borderline, or they conflict.
- The confidence is stated in both the dashboard row and the Slack alert so we know whether to trust the tag or dig.

Conditional EmailBison deep-check

Only fires when the checks above come back clean but the client is **still Critical** (i.e., "the usual suspects don't explain this, look deeper"). This gate keeps it from adding load on every run. It should surface anything we're not currently monitoring at the instance level.

Historical comparison

Where we have prior-cycle data for the client, compare current pace to their own historical cycles. Some clients are natural slow-starters who catch up — flag those with a `Historically recovers` note so we don't chase them every cycle. (First-cycle clients have no baseline — see Open Questions.)

5. Failed Campaigns + Auto-Retry (scope as fast-follow, not v1)

Two separate pieces:

- **Detecting** failed campaigns and surfacing them in the pacing panel — part of this build.
- **Auto-retrying** failed campaigns — net-new automation that does not exist yet (today it only shows in the campaigns section). This is a **write** action into EmailBison, distinct from the read-only diagnostic panel, so it should be **scoped and priced as its own line item**.

Required guardrail — do not blind-retry. If a campaign failed due to disconnected/unhealthy accounts, retrying just pushes sends into broken infrastructure and can worsen deliverability. Rule:

- Auto-retry **only if** the accounts behind the campaign are currently healthy.
 - Otherwise, **flag for manual review** instead of retrying.
-

6. Slack Alerts

Goal: clear, uncluttered, not annoying — but reliable enough that it gets checked the **same day, or next day at the latest**.

- **Critical only.** At Risk and On Track never ping; they live in the dashboard.
- **One consolidated daily digest**, not one message per client, to keep it scannable. Posted at a **fixed morning time (e.g., 8:00 AM PT)** to a dedicated channel, so it becomes a predictable daily check.

Digest format (top-level message):

 **MRL Pacing — {N} clients Critical** ({date}) • **{Client}** — {actual}/{expected} MRLs • {days left} days left • projected {proj} vs {threshold} • {root-cause tag} ({confidence}) • **Day {n} Critical** • ...

- One line per client. Full diagnostic detail lives in the **thread** under the message (or via a direct link to that client's dashboard row), so the top-level stays clean.
- **"Day N Critical" counter per client** — increments each day they stay Critical. This is what makes the daily repeat feel meaningful (a stuck client visibly escalates) rather than repetitive noise.
- **Re-sends daily** while any client is Critical. Stops automatically the moment a client downgrades out of Critical — no manual resolve.
- **All-clear line (recommended, optional toggle):** on days with zero Critical clients, post a single "✅ No clients Critical today" line. Keeps the daily-check habit intact so the

channel is trusted, at the cost of one predictable message. Can be turned off if preferred.

- Each client line links directly to the pacing panel / that client's row.
-

7. Job Cadence & Data Sources

- **Cadence:** daily run, aligned to the morning Slack digest. (Real-time is unnecessary here.) All times/reporting in **Pacific Time**.
 - **Cycle data:** the job needs each client's **billing cycle start date, cycle length, and minimum MRL threshold** as a reliable source of truth. Cycles are per-client and staggered — confirm where this lives and that it's clean.
 - **Dedupe with existing panels:** a client with "no leads in 4 days" will likely also be off-pace. Make sure the two panels don't double-alert or contradict; reference/cross-link rather than duplicate.
-

8. Open Questions / Decisions Needed (please confirm before build)

1. **Pace tolerance bands** — confirm the At Risk (10–30% behind) and Critical (>30% or unrecoverable) thresholds, or set your own.
2. **MRL definition & timing** — does pace count only confirmed quality leads, or include Undetermined/pending? Leads awaiting the Friday 5 PM determination can make a client look behind mid-week. The pacing math must match how we actually book MRLs.
3. **Business days vs. calendar days** — confirming we pace and project on business days remaining (recommended), not calendar days.
4. **Client exclusions** — paused, mid-churn, transactional, and custom-agreement clients should be suppressed from flagging. Confirm the status field / flag the job should respect.
5. **First-cycle clients** — no historical baseline and possibly still warming. Do we give them a longer early window or a "new client — expected ramp" note instead of a hard Critical?
6. **Cycle data source of truth** — confirm where cycle start, length, and MRL threshold are stored and that they're reliable per client.

7. **Slack channel + all-clear toggle** — which channel, and do we want the daily “all clear” line on or off?
8. **Auto-retry scope** — confirm auto-retry of failed campaigns is scoped/priced separately as a fast-follow, with the healthy-accounts-only guardrail.